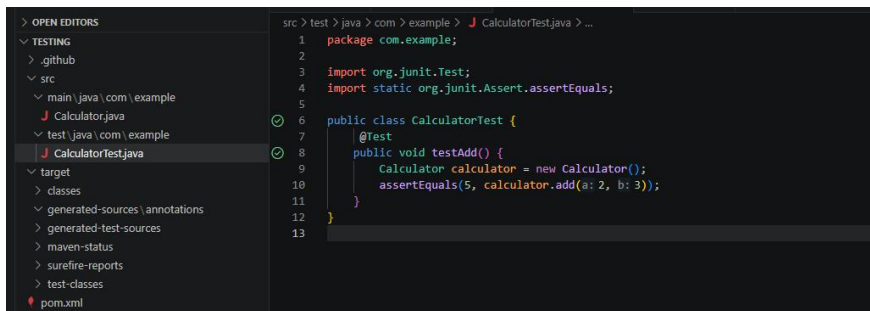


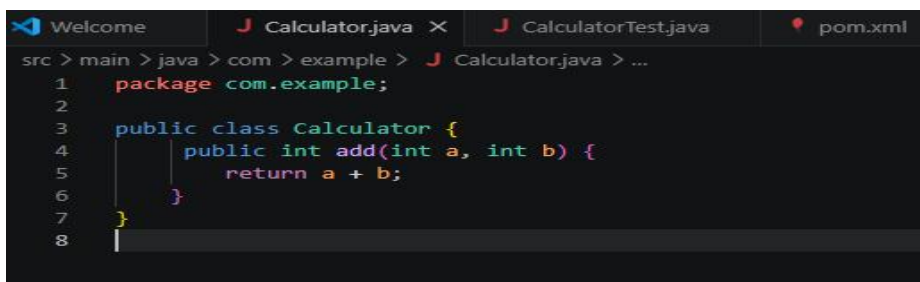
Test driven development and Logging framework

Exercise 1: Setting Up JUnit



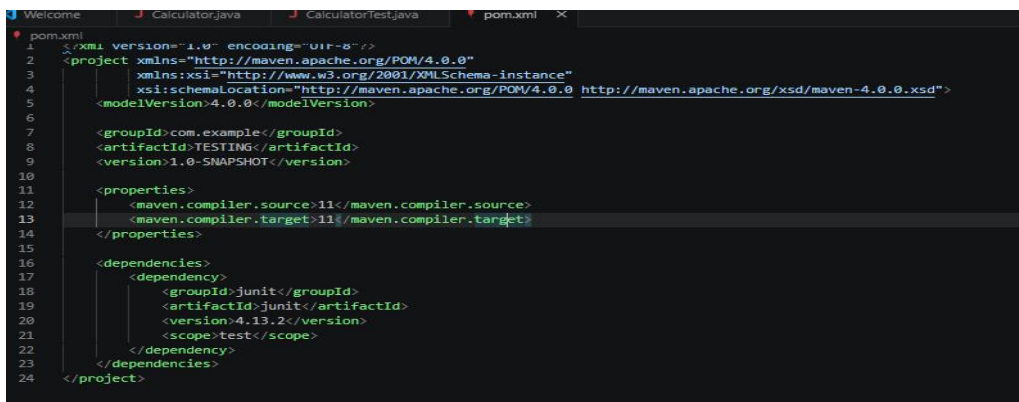
The screenshot shows an IDE with a project structure on the left and the `CalculatorTest.java` file open in the editor. The project structure includes `src`, `main`, `test`, `target`, `classes`, `generated-sources`, `generated-test-sources`, `maven-status`, `surefire-reports`, `test-classes`, and `pom.xml`. The `CalculatorTest.java` file contains the following code:

```
src > test > java > com > example > J CalculatorTest.java > ...
1 package com.example;
2
3 import org.junit.Test;
4 import static org.junit.Assert.assertEquals;
5
6 public class CalculatorTest {
7     @Test
8     public void testAdd() {
9         Calculator calculator = new Calculator();
10        assertEquals(5, calculator.add(a: 2, b: 3));
11    }
12 }
13
```



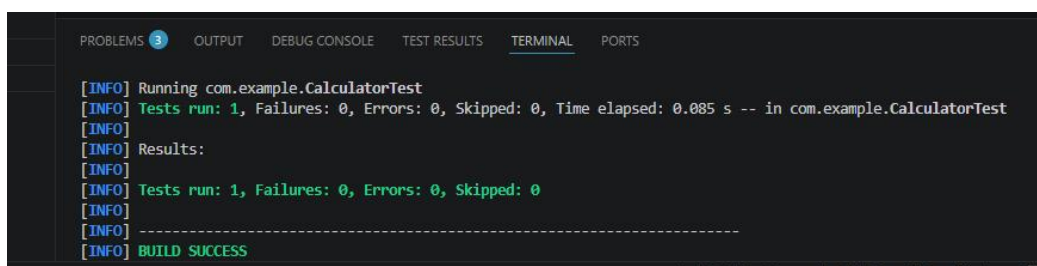
The screenshot shows an IDE with the `Calculator.java` file open in the editor. The file contains the following code:

```
Welcome J Calculator.java X J CalculatorTest.java pom.xml
src > main > java > com > example > J Calculator.java > ...
1 package com.example;
2
3 public class Calculator {
4     public int add(int a, int b) {
5         return a + b;
6     }
7 }
8
```



The screenshot shows an IDE with the `pom.xml` file open in the editor. The file contains the following code:

```
Welcome J Calculator.java J CalculatorTest.java pom.xml X
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>com.example</groupId>
8     <artifactId>TESTING</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>11</maven.compiler.source>
13         <maven.compiler.target>11</maven.compiler.target>
14     </properties>
15
16     <dependencies>
17         <dependency>
18             <groupId>junit</groupId>
19             <artifactId>junit</artifactId>
20             <version>4.13.2</version>
21             <scope>test</scope>
22         </dependency>
23     </dependencies>
24 </project>
```



The screenshot shows an IDE with the `TERMINAL` tab selected. The terminal output shows the results of a JUnit test run:

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TEST RESULTS TERMINAL PORTS
[INFO] Running com.example.CalculatorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.085 s -- in com.example.CalculatorTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
```

Exercise 2: Assertions in JUnit

```
src > test > java > com > example > J AssertionsTest.java > AssertionsTest > testAssertions()
1 package com.example;
2
3 import org.junit.Test;
4 import static org.junit.Assert.assertEquals;
5 import static org.junit.Assert.assertTrue;
6 import static org.junit.Assert.assertFalse;
7 import static org.junit.Assert.assertNull;
8 import static org.junit.Assert.assertNotNull;
9
10 public class AssertionsTest {
11
12     @Test
13     public void testAssertions() {
14         // Assert equals - checks two values are equal
15         assertEquals(5, 2 + 3);
16
17         // Assert true - checks the condition is true
18         assertTrue(5 > 3);
19
20         // Assert false - checks the condition is false
21         assertFalse(5 < 3);
22
23         // Assert null - checks the value is null
24         assertNull(null);
25
26         // Assert not null - checks the value is NOT null
27         assertNotNull(new Object());
28
29     }
30 }
```

```
20 // Assert false - checks the condition is false
21 assertFalse(5 < 3);
22
23 [INFO] Results:
24 [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
25 [INFO] BUILD SUCCESS
26 [INFO] Total time: 5.150 s
```

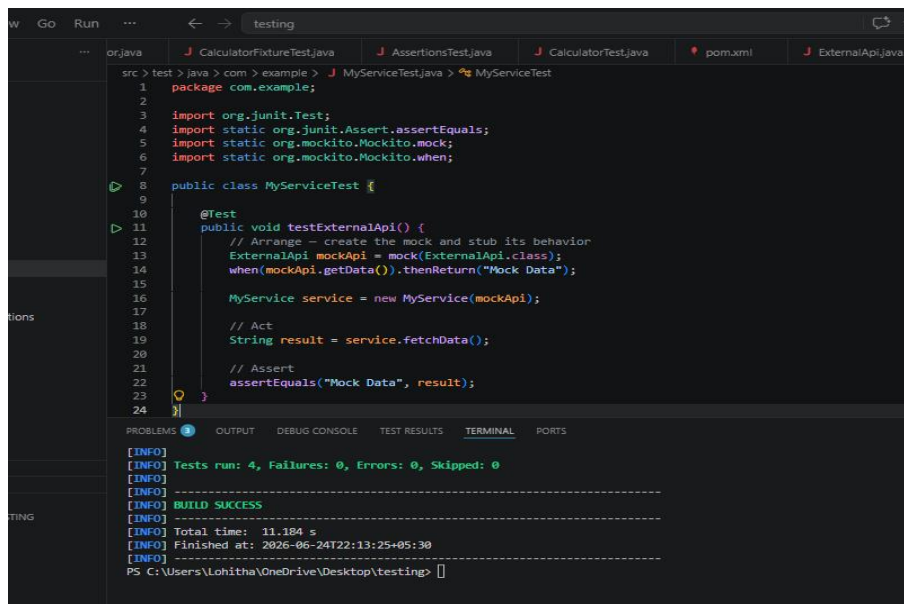
Exercise 3: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and

Teardown Methods in JUnit

```
src > test > java > com > example > J CalculatorFixtureTest.java > CalculatorFixtureTest > testAddition()
1 package com.example;
2 import org.junit.Before;
3 import org.junit.After;
4 import org.junit.Test;
5 import static org.junit.Assert.assertEquals;
6
7 public class CalculatorFixtureTest {
8
9     private Calculator calculator;
10
11     @Before
12     public void setUp() {
13         System.out.println("Setting up: creating a new Calculator instance");
14         calculator = new Calculator();
15     }
16
17     @After
18     public void tearDown() {
19         System.out.println("Tearing down: clearing Calculator instance");
20         calculator = null;
21     }
22
23     @Test
24     public void testAddition() {
25         int a = 2;
26         int b = 3;
27
28         // Act
29         int result = calculator.add(a, b);
30
31         // Assert
32         assertEquals(5, result);
33     }
34 }
35
```

```
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 5.150 s
```

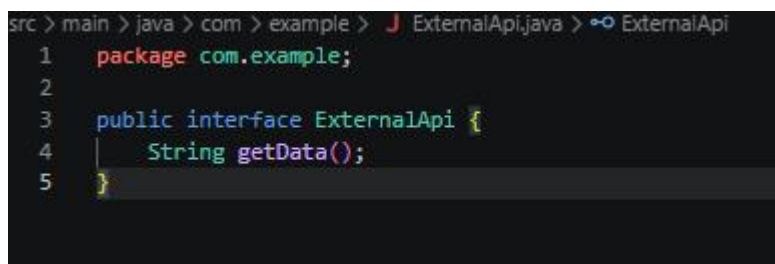
Exercise 4: Mocking and Stubbing



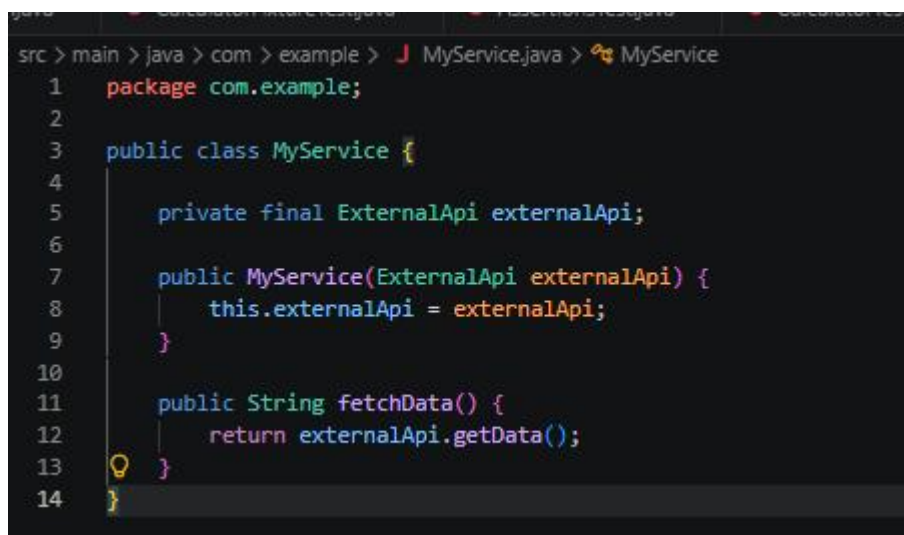
```
src > test > java > com > example > J MyServiceTest.java > MyServiceTest
1 package com.example;
2
3 import org.junit.Test;
4 import static org.junit.Assert.assertEquals;
5 import static org.mockito.Mockito.mock;
6 import static org.mockito.Mockito.when;
7
8 public class MyServiceTest {
9
10     @Test
11     public void testExternalApi() {
12         // Arrange - create the mock and stub its behavior
13         ExternalApi mockApi = mock(ExternalApi.class);
14         when(mockApi.getData()).thenReturn("Mock Data");
15
16         MyService service = new MyService(mockApi);
17
18         // Act
19         String result = service.fetchData();
20
21         // Assert
22         assertEquals("Mock Data", result);
23     }
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TEST RESULTS TERMINAL PORTS

```
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.184 s
[INFO] Finished at: 2026-06-24T22:13:25+05:30
[INFO] -----
PS C:\Users\Lohitha\OneDrive\Desktop\testing>
```



```
src > main > java > com > example > J ExternalApi.java > ExternalApi
1 package com.example;
2
3 public interface ExternalApi {
4     String getData();
5 }
```



```
src > main > java > com > example > J MyService.java > MyService
1 package com.example;
2
3 public class MyService {
4
5     private final ExternalApi externalApi;
6
7     public MyService(ExternalApi externalApi) {
8         this.externalApi = externalApi;
9     }
10
11     public String fetchData() {
12         return externalApi.getData();
13     }
14 }
```

Exercise 5: Verifying Interactions

```
src > test > java > com > example > J MyServiceVerifyTest.java > ...
1 package com.example;
2
3 import org.junit.Test;
4 import static org.mockito.Mockito.mock;
5 import static org.mockito.Mockito.verify;
6
7 public class MyServiceVerifyTest {
8
9     @Test
10    public void testVerifyInteraction() {
11        // Arrange
12        ExternalApi mockApi = mock(ExternalApi.class);
13        MyService service = new MyService(mockApi);
14
15        // Act
16        service.fetchData();
17
18        // Assert
19        verify(mockApi).getData();
20    }
21
22    @Test
23    public void testVerifyInteractionWithArguments() {
24        // Arrange
25        ExternalApi mockApi = mock(ExternalApi.class);
26        MyService service = new MyService(mockApi);
27
28        // Act
29        service.fetchDataById(id: "user123");
30
31        // Assert
32        verify(mockApi).getDataById(id: "user123");
33    }
34 }
35
```

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.example.MyServiceVerifyTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 9.412 s
[INFO] Finished at: 2026-06-24T22:22:53+05:30
[INFO]
PS C:\Users\Lohitha\OneDrive\Desktop\testing>
```

Exercise 6: Logging Error Messages and Warning Levels

```
EXPLORER
> OPEN EDITORS
  TESTING
    > .github
    > src
      > main\java\com\example
        J Calculator.java
        J ExternalApi.java
        J LoggingExample.java
        J MyService.java
      > test\java\com\example
        J AssertionsTest.java
        J CalculatorFixtureTest.java
        J CalculatorTest.java
        J MyServiceTest.java
        J MyServiceVerifyTest.java
    > target
    pom.xml

src > main > java > com > example > J LoggingExample.java > LoggingExample > main(String[])
1 package com.example;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5
6 public class LoggingExample {
7
8     private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);
9
10    Run | Debug
11    public static void main(String[] args) {
12        logger.error("This is an error message");
13        logger.warn("This is a warning message");
14    }
15 }

PROBLEMS 0 OUTPUT DEBUG CONSOLE TEST RESULTS TERMINAL PORTS
22:29:24.188 [com.example.LoggingExample.main()] WARN com.example.LoggingExample - This is a warning message
[INFO] BUILD SUCCESS
[INFO] Total time: 9.598 s
[INFO] Finished at: 2026-06-24T22:29:24+05:30
C:\Users\Lohitha\OneDrive\Desktop\testing>
```